

Here is a clean **API Documentation** for your Express backend.

Places API Documentation

Base URL

http://localhost:3000

Overview

This API manages:

- All available places
- User selected places (favorites list)

Data is stored in local JSON files:

- places.json
 - user-places.json
-

CORS Policy

All origins are allowed:

Access-Control-Allow-Origin: *

Allowed methods:

GET, PUT, DELETE

Allowed headers:

Content-Type

Endpoints

1. Get All Places

Endpoint

GET /places

Delay

- Simulated delay: **3 seconds**

Request

No body required

Response

```
{
  "places": [
    {
      "id": "p1",
      "title": "Place Name",
      "image": "image-url"
    }
  ]
}
```

Description

Returns all available places from places.json.

2. Get User Places

Endpoint

GET /user-places

Request

No body required

Response

```
{
  "places": [
    {
      "id": "p1",
      "title": "User selected place"
    }
  ]
}
```

Description

Returns places saved by the user from user-places.json.

3. Add Place to User List

Endpoint

PUT /user-places

Request Body

```
{
  "placeId": "p1"
}
```

Response

```
{
  "userPlaces": [
    {
      "id": "p1",
      "title": "Place Name"
    }
  ]
}
```

Logic

- Reads placeId
 - Finds place in places.json
 - Adds it to user-places.json **if not already present**
 - Prevents duplicates
-

Notes

- Duplicate entries are not allowed

4. Delete User Place

Endpoint

DELETE /user-places/:id

URL Parameter

Param	Type	Description
-------	------	-------------

id	string	Place ID
----	--------	----------

Response

```
{
  "userPlaces": [
    {
      "id": "p2",
      "title": "Remaining place"
    }
  ]
}
```

Logic

- Finds place by ID in user-places.json
 - Removes it if found
 - Updates file
-

5. 404 Handler

Endpoint

ALL UNKNOWN ROUTES

Response

```
{  
  "message": "404 - Not Found"  
}
```

Note

- All undefined routes return 404 JSON response
 - OPTIONS preflight requests are ignored
-

Static Files

`express.static("images")`

✓ Any file inside /images folder is publicly accessible:

`http://localhost:3000/image-name.jpg`

Tech Stack

- Node.js
 - Express.js
 - File System (fs/promises)
 - body-parser
 - JSON file storage
-

Data Flow Summary

Client

↓

Express API

↓

Reads JSON files

↓

Processes data

↓

Writes updates (for PUT/DELETE)



Returns JSON response

Key Features

- ✓ REST API design
- ✓ File-based storage (no DB)
- ✓ CORS enabled
- ✓ CRUD operations
- ✓ Async file handling
- ✓ Duplicate prevention